

JavaScript Fundamentals — Real-Life Practice Questions

Har subtopic k 4-5 practice questions. Sirf questions hain — khud solve karo. Real-world scenarios pe based hain taake pata chale ke concept kahan use hota hai.

1. JS Introduction

1. Ek e-commerce site pe "Add to Cart" button click karte hi cart count update hota hai bina page reload k — ye kis wajah se possible hai? Explain karo JS ka role.
 2. Socho tumhare paas ek static HTML page hai jo sirf text dikhata hai. Client kehta hai "form submit hone pe validation error dikhana hai instantly." JS kyun zaroori hai yahan?
 3. Ek banking dashboard load hone mein slow hai kyunke saara data server se aa raha hai. JS kaise is experience ko better bana sakta hai (client-side vs server-side thinking)?
 4. Browser console kis kaam aata hai jab tum ek live website (jaise amazon.com) ka bug debug kar rahe ho?
 5. Explain karo — agar browser mein JS disable ho jaye to ek modern web app (jaise Gmail) kaise behave karega?
-

2. JS Where To

1. Ek large project mein 500 lines ka JS code hai. Kya tum ise HTML file k andar `<script>` tag mein likhoge ya external `.js` file mein? Reasoning do.
2. Website ka loading speed slow hai kyunke script `<head>` mein top pe likha hua hai. Is issue ko `defer`/`async` ya script placement se kaise fix karoge?
3. Ek multi-page website hai jahan sabhi pages pe same navbar dropdown logic chahiye. External JS file ka approach kyun best hoga?
4. Tum ek third-party analytics script (jaise Google Analytics) ko apni site pe add kar rahe ho — usko kahan aur kaise link karoge, aur kyun uska loading order important

hai?

5. Agar do external scripts ek dusre pe depend karte hain (ek dusre ka function use karta hai), to unko HTML mein kis order mein include karoge?
-

3. JS Output

1. Ek food delivery app mein order confirm hone pe user ko alert dikhana hai "Order placed!" — konsa output method use karoge aur kyun (alert vs console vs DOM)?
 2. Debugging k dauran tum pata karna chahte ho k ek variable ki value loop k har iteration mein kya hai bina UI disturb kiye — kya approach loge?
 3. Ek registration form mein error message directly page pe (HTML element k andar) show karna hai, alert box mein nahi. Kaunsa output method appropriate hai?
 4. Client ne kaha "console mein koi debug info na dikhe production mein" — is requirement ko kaise handle karoge?
 5. Ek calculator app hai jisme result `document.write()` se print ho raha hai aur poora page clear ho jata hai dobara click pe — ye kyun hota hai aur behtar alternative kya hai?
-

4. JS Syntax

1. Ek naya developer likhta hai `let Name = 'Ali'` phir baad mein `name` use karta hai — error aata hai. Explain karo syntax rule jo yahan violate hui.
2. Payroll system mein semicolon missing hone ki wajah se ek unexpected bug aa raha hai jab do statements same line pe hain. Example bana kar dikhao.
3. Ek e-commerce cart total calculate karte waqt variable naming mein `2ndItem` likha gaya — kya ye valid hai? Reasoning do.
4. Case-sensitivity ki wajah se ek form validation function `ValidateEmail` aur `validateEmail` dono define ho gaye — real project mein isse kya masla ho sakta hai?
5. JS statements ko readable rakhne k liye indentation aur whitespace ka kya role hota hai jab team collaboratively code likhti hai?

5. JS Statements

1. Ek shopping cart total calculate karne k liye multiple statements likhne hain (price fetch karna, tax add karna, discount apply karna, final total dikhana) — inko sequence mein kaise organize karoge?
2. Statements ka execution order kyun important hai jab ek login system pehle credentials check karta hai phir dashboard load karta hai?
3. Ek attendance system mein statements galat order mein likhe gaye (pehle "present" mark hua, phir student ID check hua) — isse kya issue ho sakta hai?
4. Multiple statements ko ek single line pe semicolon se separate karna kab acceptable hota hai aur kab confusing ban jata hai (real example do)?
5. Ek weather app mein temperature conversion (Celsius to Fahrenheit) k liye statements likho jo step-by-step calculate karke result store karein.

6. JS Comments

1. Ek team project mein tumhara colleague complex regex likhta hai bina explanation k. Comments kaise us code ko maintainable banayenge?
2. Client demo se pehle tumhe temporarily ek payment gateway feature disable karna hai bina code delete kiye — konsa technique use karoge?
3. Ek large codebase mein har function k upar uska purpose, parameters, aur return value document karna hai — kis style ke comments use karoge?
4. Ek junior developer bohot zyada obvious comments likhta hai jaise `// increment i` on `i++`. Ye kyun bad practice hai? Better commenting philosophy explain karo.
5. Production code mein sensitive info (jaise API keys) comment mein reh gayi hai — ye kyun security risk hai?

7. JS Variables

1. Ek quiz app mein user ka score store karna hai jo har sahi jawab pe increase hoga — variable declare karke logic socho.

2. Ek hotel booking system mein "available rooms" ki count baar baar change hoti hai (booking aur cancellation pe). Kaunsa variable type use karoge?
 3. Tumhare paas ek user ka naam hai jo login k baad kabhi change nahi hoga session mein — kya isko variable mein store karna sahi hoga ya constant mein?
 4. Ek expense tracker app mein multiple variables (income, expense, balance) manage karne hain — inko naming convention k hisaab se kaise organize karoge?
 5. Undeclared variable use karne se ek e-commerce checkout process mein silent bug aa raha hai — kaise identify aur prevent karoge?
-

8. JS Let

1. Ek for-loop mein counter variable (`let`) se declare kiya gaya hai — iski block-scope nature real project mein kyun useful hai (var k comparison mein)?
 2. Ek OTP verification system mein temporary variable chahiye jo sirf ek function k andar valid rahe — (`let`) kyun best choice hai?
 3. Socho ek nested if-else block mein same naam ka variable different scopes mein declare karna hai (jaise `total` inner aur outer block dono mein) — (`let`) ye kaise handle karta hai?
 4. Ek bug report aaya hai jahan loop k andar (`var`) use hone ki wajah se async callback mein galat value capture ho rahi hai. (`let`) isse kaise fix karega? Example do.
 5. E-commerce filter system mein user ka selected category temporarily store karna hai jo page refresh pe reset ho jaye — (`let`) ka use case explain karo.
-

9. JS Const

1. Ek app mein tax rate (jaise 17% GST) fix hai aur kabhi change nahi hogi runtime pe — isko declare karne ka best tareeqa kya hai?
2. Ek array (jaise cart items list) (`const`) se declare kiya gaya hai lekin uske andar items push/remove ho rahe hain — ye kaise possible hai jab const "constant" hai?
3. API base URL (jaise `https://api.example.com`) poore project mein use hoti hai — (`const`) kyun is k liye ideal hai?

4. Ek developer galti se `const` variable ko reassign karne ki koshish karta hai aur error aata hai — real scenario do jahan ye protection helpful ho.
 5. Ek configuration object (theme colors, app settings) banate waqt `const` vs `let` ka decision kaise loge?
-

10. JS Types

1. Ek form se user ki age input string k roop mein aati hai ("25"). Isko number type mein convert kiye bina agar calculation ki jaye to kya masla hoga?
 2. Ek user profile object mein naam (string), age (number), isPremiumMember (boolean), aur hobbies (array) hain — har field ka data type identify karo aur reasoning do.
 3. API response mein ek field `null` aa rahi hai lekin frontend usse `undefined` treat kar raha hai — dono mein practical difference kya hai, real bug example do.
 4. Ek payment app mein `typeof` operator ka use karke input validate karna hai (ke amount number hi ho) — kaise likhoge?
 5. Dynamic typing ki wajah se ek calculator app mein `"5" + 3` ka result "53" aa raha hai instead of 8 — ye kyun hota hai aur kaise fix karoge?
-

11. JS Operators

1. Ek ride-sharing app mein total fare calculate karna hai (base fare + distance*rate + tax). Kaunse operators use honge is expression mein?
2. Discount system mein agar price 1000 se zyada hai to 10% discount, warna 5% — operators combine karke kaise likhoge?
3. Ek inventory system mein stock quantity update karni hai (increment/decrement) jab order place ya cancel ho — kaunse operators helpful hain?
4. Login system mein username aur password dono correct hone chahiye tabhi access mile — kaunsa logical operator use hoga?
5. Ek age-restricted content app mein user ki age 18 ya usse zyada honi chahiye — comparison operator kaise apply karoge?

12. JS Arithmetic

1. Ek restaurant bill splitting app mein total amount ko equally N logon mein divide karna hai — division operator ka real use case likho.
2. E-commerce mein bulk order pe per-unit price calculate karna hai (total price / quantity) — floating point precision ka issue kaise handle karoge?
3. Ek inventory system mein remaining stock check karna hai ke wo kisi fixed pack size (jaise 12) ka multiple hai ya nahi — kaunsa operator use hoga?
4. Salary calculation mein overtime hours ka pay alag rate se add karna hai — arithmetic expression banao.
5. Ek age calculator app mein current year se birth year subtract karke age nikalni hai — is mein kya edge cases (jaise birthday abhi nahi aayi) consider karoge?

13. JS Assignment

1. Ek shopping cart mein total price ko baar baar update karna hai jab naya item add ho — `+=` operator kaise use hoga?
2. Ek game mein player ki health decrease honi hai jab damage lage — konsa compound assignment operator use karoge?
3. Loyalty points system mein points ko double karna hai kisi promotion k dauran — `*=` ka practical example do.
4. Ek counter app mein reset button click hone pe value 0 pe wapis assign karni hai — simple assignment ka use case.
5. Bank account balance se withdrawal amount minus karna hai har transaction pe — assignment operator se real logic likho.

14. JS Comparisons

1. Login form mein user input `"123"` (string) aur stored password `123` (number) compare ho raha hai — `==` aur `===` ka result alag kyun hoga? Explain karo.

2. Ek age verification system mein `age >= 18` check karna hai — real scenario mein isko kaise implement karoge?
 3. E-commerce mein do product prices compare karke "cheaper" wala highlight karna hai — comparison operators ka use.
 4. Ek quiz app mein user ka answer expected answer se match karna hai (case-sensitive ya insensitive dono scenarios socho) — `===` kyun preferred hai `==` se?
 5. Stock management mein `quantity !== 0` check karke "in stock" ya "out of stock" dikhana hai — likho kaise.
-

15. JS Conditional (Overview)

1. Ek weather app mein temperature k hisaab se message dikhana hai ("Hot", "Moderate", "Cold") — conditional logic ka structure socho.
 2. Ek discount system mein multiple conditions hain (membership type, order amount, coupon code) — inko kaise combine karoge?
 3. Traffic light simulation mein color k hisaab se action decide karna hai (Red = Stop, Green = Go, Yellow = Slow) — conditional approach do.
 4. Ek loan approval system mein credit score, income, aur existing debt teen factors pe depend karta hai — conditions kaise design karoge?
 5. Real-life scenario: ek ATM machine withdrawal amount check karta hai balance se zyada na ho — is logic ko conditional statements mein socho.
-

16. JS If

1. Ek exam result system mein agar marks 40 se kam hain to "Fail" print karna hai — simple `if` statement likho.
2. Ek e-commerce site pe agar cart empty hai to "checkout" button disable karna hai — condition socho.
3. Weather app mein agar temperature 40°C se zyada hai to "Heat warning" show karna hai — likho.

4. Login attempt system mein agar user 3 se zyada baar galat password dale to account lock karna hai — `if` ka use case.
 5. Ek parking system mein agar available slots 0 hain to "Parking Full" dikhana hai — likho.
-

17. JS If Else

1. Ek age verification system mein 18+ ho to "Access granted" warna "Access denied" dikhana hai — `if-else` likho.
 2. E-commerce checkout mein agar payment successful ho to "Order Confirmed" warna "Payment Failed" — logic banao.
 3. Ek attendance system mein agar student 75% se zyada present hai to "Eligible for exam" warna "Not eligible" — likho.
 4. Grading system mein multiple ranges hain (A, B, C, F) — `if-else if-else` chain kaise banaoge?
 5. Ek subscription app mein agar trial period khatam ho gaya hai to payment page dikhana hai warna app access dena hai — likho.
-

18. JS Ternary

1. Ek UI mein user login hai ya nahi uske hisaab se "Logout" ya "Login" button text dikhana hai — ternary operator se likho.
 2. Discount eligibility check (order > 5000 ho to 10% discount warna 0%) — ternary se short likho.
 3. Ek form mein required field empty hai ya nahi uske hisaab se border color red ya green karna hai — ternary use case.
 4. Stock status ("In Stock" / "Out of Stock") ek single line mein dikhana hai based on quantity — ternary likho.
 5. Ek theme switcher app mein dark mode on/off k hisaab se background color set karna hai — ternary se implement karo.
-

19. JS Switch

1. Ek days-of-week app mein number (1-7) k hisaab se din ka naam dikhana hai — `switch` statement likho.
 2. E-commerce mein payment method (Cash, Card, EasyPaisa, JazzCash) k hisaab se alag processing logic chalana hai — switch se karo.
 3. Ek traffic light system mein color k hisaab se action decide karna hai (multiple if-else ki jagah switch use karke).
 4. Grading system (A, B, C, D, F) mein switch se message dikhana hai — likho, aur `default` case ka role explain karo.
 5. Ek calculator app mein operator symbol (+, -, *, /) k hisaab se operation perform karni hai — switch case likho.
-

20. JS Booleans

1. Ek to-do list app mein har task ka "completed" status track karna hai — boolean variable kaise use karoge?
 2. Login system mein `isLoggedIn` flag kaise poore app mein different pages pe access control k liye use hoga?
 3. Ek e-commerce product mein "inStock" boolean k hisaab se "Add to Cart" button enable/disable karna hai — likho.
 4. Form validation mein multiple fields valid hain ya nahi track karne k liye boolean flags kaise use karoge?
 5. Ek dark mode toggle feature banani hai — boolean state kaise manage karoge?
-

21. JS Logical

1. Ek login system mein username AND password dono correct hone chahiye — `&&` operator ka real use.
2. Ek discount system mein agar user premium member HAI ya order 5000 se zyada HAI to discount milega — `||` operator use karo.

3. Ek form submit button tab tak disabled rahe jab tak saari required fields filled NAHI hain — `!` (NOT) operator ka use case.
 4. E-commerce filter mein price range AND category dono match karne chahiye — logical AND ka practical example.
 5. Ek notification system mein agar user ne notifications on ki hain AND app background mein nahi hai to alert bhejni hai — combine logical operators.
-

22. JS Loops (Overview)

1. Ek shopping cart mein saare items ki total price calculate karni hai — loop kyun zaroori hai yahan (single variable ki jagah)?
 2. Ek attendance system mein 30 students ki list pe har ek ka status check karna hai — loop se kaise iterate karoge?
 3. Ek social media app mein user k saare posts ek ek karke load karne hain — real-life loop use case.
 4. Multiplication table print karni hai 1 se 10 tak — loop socho.
 5. Ek inventory system mein har product ka stock check karke low-stock items alag list mein dalne hain — loop logic socho.
-

23. JS Loop for

1. Ek e-commerce cart mein `for` loop se saare items ka naam aur price console pe print karo.
 2. 1 se 100 tak sirf even numbers print karne hain — `for` loop likho.
 3. Ek attendance sheet mein 30 din ka data hai, `for` loop se present days count karo.
 4. Reverse counting (10 se 1 tak) ek countdown timer k liye — `for` loop se likho.
 5. Ek matrix (2D array, jaise seat booking layout) ko `for` loop (nested) se traverse karo.
-

24. JS Loop while

1. Ek ATM machine mein user se PIN barbar poochna hai jab tak sahi na ho — `while` loop kyun better fit hai `for` se?
 2. Game mein player ki health 0 se zyada hai jab tak game chalta rahe — `while` loop se likho.
 3. Ek download progress simulation mein jab tak progress 100% na ho, loop chalta rahe — likho.
 4. User input validate karna hai jab tak valid email na de — `while` loop ka use case.
 5. `do-while` aur `while` mein difference ek real scenario se explain karo (jaise ek menu jo kam se kam ek baar zaroor dikhe).
-

25. JS Break

1. Ek search function mein array mein item mil jaye to loop ko turant rokna hai — `break` kaise use hoga?
 2. Ek login system mein 3 galat attempts k baad loop se bahar nikal jana hai — likho.
 3. Nested loop (jaise seat search 2D array mein) mein sirf inner loop se break karna hai — kaise handle karoge (labeled break bhi socho)?
 4. Ek inventory check mein pehla out-of-stock item milte hi loop stop karna hai — `break` ka use.
 5. Game mein "Game Over" condition milte hi loop terminate karna hai — real example likho.
-

26. JS Continue

1. Ek attendance loop mein absent students ko skip karke sirf present students process karne hain — `continue` kaise use hoga?
2. E-commerce mein out-of-stock products ko listing loop mein skip karna hai — likho.

3. Ek number processing loop mein sirf even numbers pe operation karni hai, odd ko skip karna hai — `continue` se likho.
 4. Form data validation loop mein empty fields ko skip karke sirf filled fields process karni hain.
 5. Ek spam filter mein blacklisted emails ko skip karke baaki process karne hain — `continue` ka real use case.
-

27. JS Control Flow

1. Ek e-commerce checkout process mein multiple steps hain (cart validate → address check → payment → confirmation) — control flow kaise design karoge taake har step sahi order mein chale?
 2. Error handling k sath control flow — agar payment fail ho to age na badhein, wapis payment step pe le jayein — socho.
 3. Ek quiz app mein agar time khatam ho jaye to loop turant break ho aur result show ho — control flow combine karo (loop + break + condition).
 4. Nested conditions aur loops ka combination — ek seat booking system jo rows aur columns dono check kare — control flow socho.
 5. Real-life analogy do jahan sequential, conditional, aur iterative control flow teeno ek hi feature (jaise online exam system) mein use hote hain.
-

28. JS Strings

1. Ek user ka full naam "Ali Khan" hai, usse first name aur last name alag karne hain — strings pe kaam socho.
2. Ek chat app mein message length 200 characters se zyada na ho, validate karna hai — string property use karoge.
3. Email format mein "@" symbol hona zaroori hai — string check karna hai.
4. Product description mein extra spaces (leading/trailing) remove karne hain before saving — konsa concept use hoga?

5. Username create karte waqt user "Ali@123" likhta hai — special characters allow honge ya nahi, kaise check karoge?
-

29. JS String Templates

1. Ek order confirmation message banani hai: "Hi Ali, your order #1234 of Rs. 2500 has been placed." — template literals se dynamic values kaise insert karoge?
 2. Multi-line invoice text (address, items, total) banani hai — template literals ka multi-line feature kyun useful hai (concatenation k comparison mein)?
 3. Ek greeting message app mein time k hisaab se ("Good Morning/Afternoon/Evening") aur user ka naam combine karna hai — template string se likho.
 4. Product card mein price ko currency format mein show karna hai ("₹" + price) — template literal se karo aur (+) concatenation se compare karo.
 5. Ek dynamic URL banani hai (jaise `https://api.example.com/users/123/orders`) jisme user ID variable se aati hai — template literal se likho.
-

30. JS String Methods

1. User ka input hamesha lowercase mein store karna hai (email consistency k liye) — konsa method use karoge?
 2. Ek search feature mein user "laptop" search kare to product names mein "Laptop" bhi match hona chahiye — case-insensitive search kaise karoge?
 3. Long product description ko card pe sirf pehle 100 characters dikhane hain aur "..." add karna hai — konsa method chahiye?
 4. Ek CSV file se aayi string ko comma se split karke array banana hai — method socho.
 5. User ka phone number string mein spaces/dashes hain (jaise "0300-1234567") — inko remove karke sirf digits rakhne hain, kaise karoge?
-

31. JS String Search

1. Ek content moderation system mein message mein koi banned word hai ya nahi check karna hai — konsa method use karoge?
 2. Search bar mein user "iphone" likhta hai, product list mein us word ka position pata karna hai (match kahan start hota hai) — method socho.
 3. Ek email validation mein check karna hai ke string ".com" ya ".pk" pe end hoti hai ya nahi.
 4. File upload system mein filename ".jpg" ya ".png" se start/end hoti hai check karna hai.
 5. Ek URL router mein path "/admin" se start hoti hai to alag logic chalana hai — string search method se check karo.
-

32. JS String Reference

1. Ek project mein tumhe yaad nahi ke kaunsa method string ko uppercase karta hai — kaise reference dhoondoge aur use karoge (`toUpperCase()` example do)?
 2. `padStart()` method real project mein kab use hota hai — ek invoice number formatting example do (jaise "007").
 3. `repeat()` method ka ek practical use case socho (jaise loading dots "...").
 4. `trim()`, `trimStart()`, `trimEnd()` mein difference ek form input cleaning scenario se explain karo.
 5. Ek reference table banao (apne notes k liye) jisme 5 string methods aur unka ek-ek real use case ho.
-

33. JS Numbers

1. Ek e-commerce price field mein user "abc" type kar deta hai — isko number validate kaise karoge?
2. Currency amount ko 2 decimal places tak round karna hai (jaise 99.999 → 100.00) — kaunsa concept use hoga?

3. Ek temperature sensor se float value aa rahi hai (36.66666) — display k liye usko clean karna hai.
 4. Large numbers (jaise 1000000) ko readable format mein dikhana hai (1,000,000) — numbers ka concept socho.
 5. Ek age calculator mein negative number input ho jaye to usse invalid treat karna hai — kaise check karoge?
-

34. JS Number Methods

1. Product price ko fixed 2 decimals mein dikhana hai ("99.50" not "99.5") — konsa method use karoge?
 2. Ek form validation mein check karna hai ke input value actually ek valid number hai — method socho.
 3. User se input string "42" aata hai jo calculation mein use karna hai — number mein convert karne k methods (parseInt/parseFloat) ka farak samjhao real example se.
 4. Ek scientific calculator app mein number ko string representation mein convert karna hai display k liye — method socho.
 5. Ek infinite loop bug se `Infinity` value aa gayi hai calculation mein — kaise detect aur handle karoge?
-

35. JS Number Properties

1. Ek calculator app mein sabse bada aur chhota safe number kya hota hai JS mein — real limitation kab face hogi (jaise bank ka bohot bada transaction amount)?
2. `Number.MAX_SAFE_INTEGER` se zyada value store karne pe kya masla aa sakta hai — ek real-life scenario socho (jaise unique ID generation).
3. Division by zero ka result kya hota hai aur ek real app (jaise average calculator jab entries 0 hon) mein isse kaise handle karoge?
4. `NaN` (Not a Number) real project mein kab aata hai — ek form calculation ka example do jahan ye bug create kar sakta hai.

5. Ek app mein floating point precision issue hai ($0.1 + 0.2 \neq 0.3$) — real payment calculation mein isse kaise avoid karoge?
-

36. JS Number Reference

1. Ek reference table banao jisme 5 number methods/properties aur unka real-world use case ho (apne notes k liye).
 2. `Number.isInteger()` ka ek practical use socho (jaise quantity field jahan decimal allowed nahi honi chahiye).
 3. `toLocaleString()` method currency ya date formatting mein kaise kaam aati hai — example do.
 4. Ek unit converter app (km to miles) banate waqt kaunse number methods reference karoge?
 5. Number formatting (currency, percentage) ek dashboard app mein consistently kaise maintain karoge across multiple components?
-

37. JS Bitwise

1. Ek permissions system (jaise read=1, write=2, execute=4) mein user k multiple permissions ek hi number mein store karne hain — bitwise OR ka real use case socho.
 2. Check karna hai ke ek user k paas specific permission hai ya nahi (jaise "write" access) — bitwise AND kaise use hoga?
 3. Ek game mein character ki multiple abilities (flags) ek single integer mein pack karni hain memory efficiency k liye — bitwise operators ka role.
 4. Fast even/odd number check karna hai (performance-critical loop mein) — bitwise AND (`& 1`) se kaise karoge, normal modulo se compare karo.
 5. Color values (RGB) ko bitwise shift operators se combine/extract karna hai ek image processing app mein — socho kaise.
-

38. JS BigInt

1. Ek cryptocurrency wallet app mein transaction amounts itni badi hain ke normal `Number` type precision khatam kar deta hai — `BigInt` kyun zaroori hai yahan?
 2. Ek unique ID generator system 64-bit se bade IDs generate karta hai — `BigInt` ka use case socho.
 3. `BigInt` aur normal `Number` ko directly compare/mix karne ki koshish karo — kya error aata hai aur kyun (real bug scenario)?
 4. Ek scientific computing app mein factorial calculate karna hai bade numbers (jaise 25!) k liye — `BigInt` kyun zaroori hoga normal Number k comparison mein?
 5. Banking system mein total national debt jaisi extremely large value store karni hai — `BigInt` literal (`123n`) kaise likhoge aur kab use karoge?
-
-
-

Part 2 — Coding Exercises: Real-World Mini Systems

Ye sirf syntax practice nahi hai — har section mein tum ek **chhota real system bana rahe ho**, aur usmein wahi problems aayengi jo actual production code mein aati hain (bad input, edge cases, wrong data type, empty data, precision issues waghera). Requirements ko poora karo, phir khud edge cases test karo jo diye gaye hain.

Project 1: Order ID & Logger System

(Covers: Intro, Syntax, Statements, Comments, Variables, Let, Const)

Tum ek e-commerce backend ka chhota logging module bana rahe ho.

1. `generateOrderId()` function likho jo har call pe ek naya unique order ID return kare (jaise "ORD-1001", "ORD-1002"...). **Problem:** counter ko kahan store karoge k wo function calls k darmiyan reset na ho? (`let` vs `const`) ka sahi decision socho — outer scope counter.)
2. `logEvent(type, message)` function likho jo console pe `[timestamp] [type] message` format mein print kare, jaise ek real app ka logger karta hai.

3. Ek `STORE_CONFIG` object `const` se banao (storeName, taxRate, currency) jo poore module mein reuse ho. **Problem:** agar koi galti se `STORE_CONFIG.taxRate = 999` kar de to kya hoga — kaise prevent karoge partial protection dekar (`Object.freeze()` explore karo)?
 4. Test karo: `generateOrderId()` ko 5 baar call karo — agar IDs duplicate ya wrong order mein aayen to bug hai, fix karo.
-

Project 2: User Registration Validator

(Covers: Types, Comparisons, If/Else, Ternary, Booleans, Logical Operators)

Ek signup form ka backend validation likho jahan raw input aata hai (aksar galat type mein).

1. `validateAge(input)` function likho jo form se aane wala string (jaise `"25"`) le, number mein convert kare, aur check kare `age >= 18`. **Problem:** agar input `"abc"` ho, `""` ho, ya negative number ho to kya hoga? Sab cases handle karo (`isNaN`, empty check).
 2. `isStrongPassword(password)` function likho jo check kare: length ≥ 8 , kam se kam ek number, kam se kam ek uppercase letter — sab conditions `&&` se combine karo.
 3. `canRegister(age, email, password)` function likho jo teeno validations combine kare aur ya to `true` return kare ya specific error message (string) — "kaunsa field fail hua" pata chalna chahiye, generic "Invalid" nahi.
 4. **Real bug scenario:** ek user email `"ALI@GMAIL.COM"` se signup karta hai, dusra `"ali@gmail.com"` se — system dono ko alag user samajh raha hai. Fix likho (case-insensitive comparison).
-

Project 3: Ride Fare & Discount Calculator

(Covers: Operators, Arithmetic, Assignment, Switch)

Ek ride-sharing app (jaise Careem/Bykea) ka fare engine bana rahe ho.

1. `calculateFare(distanceKm, rideType)` function likho jo `switch` se `rideType` ("Bike", "Car", "Rickshaw") k hisaab se alag base fare aur per-km rate apply kare.
 2. Peak hours mein 1.5x surge pricing lagti hai — function mein ek `isPeakHour` boolean parameter add karo jo total pe surge multiply kare.
 3. **Problem:** distance 0 ya negative aa jaye (GPS error ki wajah se) to fare calculate nahi hona chahiye — invalid input handle karo aur meaningful error return karo.
 4. **Precision problem:** fare `19.9999999999998` jaisa float aa raha hai — real currency display k liye isko kaise fix karoge (2 decimals tak round)?
 5. Test: 5 alag rides (different types, distances, peak/off-peak) k liye fare calculate karke result table console pe print karo.
-

Project 4: Attendance & Inventory Tracking System

(Covers: Loops — *for/while*, *Break*, *Continue*, *Control Flow*)

Ek school attendance system aur ek warehouse inventory system dono ki logic banani hai.

1. `countPresentDays(attendanceArray)` function likho — array mein `"P"`/`"A"` values hain (30 din ka data), `for` loop se present days count karo.
2. `findFirstLowStock(inventory)` function likho — array of `{name, quantity}` objects hai, loop se pehla item dhoondo jiski quantity 10 se kam hai, milte hi `break` karke turant return karo (poora array traverse na karo — real performance concern).
3. `getActiveUsersReport(users)` function likho jo `users` array (har user mein `isActive` field hai) mein se inactive users ko `continue` se skip kare aur sirf active users ka naam print kare.
4. `simulatePinAttempts(correctPin, attemptsArray)` function likho jo `while` loop se attempts array se ek ek PIN check kare — sahi milte hi rukh jaye, 3 galat attempts k baad "Locked" return kare.
5. **Problem — infinite loop bug:** ek `while` loop likha gaya hai jahan counter increment karna bhool gaye — is bug ko identify karne ka tareeqa aur real app mein

iske consequences (browser freeze) discuss karo, phir fix karo.

Project 5: Search & Text Processing Engine

(Covers: Strings, Templates, String Methods, String Search)

Ek chhota product-search aur content-moderation engine bana rahe ho.

1. `searchProducts(query, productList)` function likho jo case-insensitive search kare — user "IPHONE" ya "iphone" likhe, dono se match milna chahiye (`toLowerCase()` + `includes()`).
 2. `generateInvoiceText(customerName, items, total)` function likho jo template literals se ek multi-line formatted invoice string banaye (real receipt jaisi).
 3. `containsProfanity(message, bannedWordsList)` function likho jo check kare message mein koi bhi banned word hai — **problem:** "classic" mein "lass" jaisa substring match galti se trigger na ho, is false-positive ko kaise avoid karoge (word boundary soch kar, split/includes se)?
 4. `truncateProductDescription(desc, maxLen)` function likho jo lambi description ko cut kare aur "..." laga de, lekin **problem:** beech mein se word cut na ho (jaise "This is amaz..." nahi, poora "This is..." tak hona chahiye) — word-boundary pe hi truncate karo.
 5. Test: 10 fake product names ka array banao, 3 different search queries test karo (exact match, partial match, no match) aur result verify karo.
-

Project 6: Pricing Engine with Precision & Large Numbers

(Covers: Numbers, Number Methods, Number Properties, Bitwise, BigInt)

Ek e-commerce checkout ka final pricing engine — jahan real money precision issues aate hain.

1. `calculateCartTotal(items)` function likho — items array mein `{price, qty}` hai, total nikalo. **Problem:** floating point addition (jaise $0.1 + 0.2$) ki wajah se total mein

"0.300000000000000004" jaisi galat value aa sakti hai — kaise fix karoge (cents mein calculate karke baad mein divide karna ek common real-world trick hai)?

2. `formatCurrency(amount)` function likho jo hamesha 2 decimal places return kare, chahe input `99`, `99.5`, ya `99.999` ho.
3. `isValidQuantity(input)` function likho jo bitwise trick (`& 1`) nahi, balke `Number.isInteger()` se check kare quantity ek whole number hai (decimals allowed nahi order quantity mein).
4. `hasPermission(userPermissions, requiredPermission)` function likho jo bitwise `&` operator se permission flags check kare (jaise `read=1`, `write=2`, `delete=4` — ek user k paas `read+write=3` hai, check karo "write" allowed hai ya nahi).
5. **Large number problem:** ek transaction ID system bana rahe ho jahan IDs `Number.MAX_SAFE_INTEGER` (9007199254740991) se aage nikal gaye hain — `BigInt` use karke ek naya ID generator likho jo safely bade numbers handle kare, aur dikhao ke normal `Number` is case mein kyun fail hota (precision loss test karke).

Final Integration Challenge: Mini E-Commerce Checkout System

Ab sab kuch combine karo — ek chhota lekin real checkout flow function chain banao:

1. `validateCart(cart)` — empty cart check, invalid quantity check (loops + conditions)
2. `applyDiscount(total, user)` — membership aur order value k hisaab se discount (logical operators + ternary)
3. `calculateTax(total, province)` — switch se different provinces ka alag tax rate
4. `generateReceipt(cart, user, total)` — template literals se poori receipt string
5. `processCheckout(cart, user)` — upar k saare functions ko sahi order mein call kare aur agar koi step fail ho (jaise empty cart) to age na badhe, meaningful error return kare

Test cases design karo khud: empty cart, invalid quantity (negative/decimal), no discount eligible, multiple provinces, aur ek "happy path" jahan sab kuch sahi ho.

Bonus: Mixed Real-World Mini Challenges

1. **To-Do List Logic:** Variables, loops, conditions, aur string methods combine karke ek to-do app ka core logic socho (add task, mark complete, filter pending).
2. **Grading System:** Switch/if-else, arithmetic, aur numbers combine karke student grades calculate karne ka system design karo.
3. **Shopping Cart Total:** Loops, arithmetic, template literals combine karke cart summary (items, subtotal, tax, total) banane ka logic likho.
4. **Login Attempt Limiter:** Loops, break, booleans, aur logical operators combine karke 3-attempt login lockout system socho.
5. **Password Strength Checker:** String methods, logical operators, aur conditions combine karke password validation (length, uppercase, number, special char) ka logic design karo.

Total: 38 topics $\times$ 4-5 questions + 5 bonus mixed challenges